

# **Software-In-the-Loop (SIL) & Serial Port Calculator**

Babak Hajari

<https://www.linkedin.com/in/babak-hajari-a20095105/>

<https://github.com/BabakHajari/Software-in-the-Loop-SIL-Modbus-RTU-Serial-Port>

Montreal, Canada

September 2026

© Babak Hajari, 2026

## **Abstract**

This document presents the Software-In-the-Loop (SIL) & Serial Port Calculator, a software tool designed to simulate industrial devices connected through serial communication ports such as RS-232 and RS-485. The application is intended for PLC programmers, automation engineers, and software developers who need to test communication, command handling, and data processing when the real machine or device is unavailable, difficult to access, or requires special operating conditions.

The document introduces the fundamentals of the Modbus protocol, including the master–slave communication model, Modbus RTU structure, function codes, data fields, CRC error checking, and exception responses. It also demonstrates practical Modbus communication using Advantech ADAM modules, explaining how request and response messages are structured and interpreted.

In addition to Modbus communication concepts, the document explains how the SIL software can simulate device behavior by receiving predefined commands and automatically sending corresponding responses. It describes the use of virtual serial ports, command/response configuration through the MessageSIL.ini file, automatic CRC generation, configurable response delay, loop-based operation, logging, and conversion tools for hexadecimal, decimal, IEEE floating-point, real-value, and string data.

Overall, the software provides a practical SIL environment for testing Modbus RTU and normal serial communication without requiring physical hardware. It is useful for industrial automation, PLC-to-device communication, building automation, energy management, data acquisition, hardware simulation, and development-stage validation of control systems.

## Acknowledgments

The development of the Software-In-the-Loop (SIL) & Serial Port Calculator was motivated by the practical needs of PLC programmers, automation engineers, and software developers who require a simulation environment for testing serial communication without relying on the continuous availability of physical machines or industrial devices. The author acknowledges the importance of Software-In-the-Loop methodology in enabling communication testing, data processing validation, and control-logic verification under conditions where real hardware may be unavailable, difficult to access, or impractical to operate.

Special acknowledgment is given to the industrial automation community and users of RS-232/RS-485 communication systems, whose real-world troubleshooting and testing requirements inspired the inclusion of practical features such as configurable serial port settings, predefined command/response handling, Modbus RTU CRC calculation, response-delay configuration, hexadecimal and decimal conversion, IEEE floating-point conversion, and real-value interpretation.

The document also recognizes the value of practical examples based on Advantech ADAM modules, including ADAM-4017, ADAM-4015, and ADAM-4018, which help demonstrate how predefined Modbus and normal serial messages can be configured, received, interpreted, and answered by the SIL software during simulation.

Finally, appreciation is extended to users who test the software, report issues, and provide feedback for future improvement. Their input contributes to improving the reliability, usability, and functionality of the software package as a practical tool for Modbus RTU, serial-port simulation, and industrial communication testing.

## Table of Contents

|                                                                                                                                                                                                                                                                                                                               |     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Software-In-the-Loop (SIL) & Serial Port Calculator .....                                                                                                                                                                                                                                                                     | 0   |
| Abstract.....                                                                                                                                                                                                                                                                                                                 | II  |
| Acknowledgments .....                                                                                                                                                                                                                                                                                                         | III |
| Table of Contents .....                                                                                                                                                                                                                                                                                                       | IV  |
| List of Abbreviations .....                                                                                                                                                                                                                                                                                                   | V   |
| List of Figures .....                                                                                                                                                                                                                                                                                                         | VI  |
| 1. Introduction .....                                                                                                                                                                                                                                                                                                         | 1   |
| 1.1. Introduction.....                                                                                                                                                                                                                                                                                                        | 2   |
| The Software-In-the-Loop (SIL) and Serial Port Calculator is a software tool for PLC<br>programmers and engineers who need a simulation environment with serial communication<br>ports (RS-232/RS-485) to respond on behalf of real devices. It provides practical utilities for<br>industrial communication, including:..... | 2   |
| 1.2. Overview of the Modbus Protocol.....                                                                                                                                                                                                                                                                                     | 2   |
| 1.3. Modbus Communication Model.....                                                                                                                                                                                                                                                                                          | 2   |
| 1.4. Modbus RTU .....                                                                                                                                                                                                                                                                                                         | 2   |
| 1.5. Modbus Data Model .....                                                                                                                                                                                                                                                                                                  | 3   |
| 1.6. Modbus RTU Message Frame .....                                                                                                                                                                                                                                                                                           | 3   |
| 1.6.0. Field Descriptions:.....                                                                                                                                                                                                                                                                                               | 3   |
| 1.6.1. Address (1 byte) .....                                                                                                                                                                                                                                                                                                 | 3   |
| 1.6.2. Function Code (1 byte).....                                                                                                                                                                                                                                                                                            | 3   |
| 1.6.3. Data Field (N bytes) .....                                                                                                                                                                                                                                                                                             | 3   |
| 1.6.4. CRC (2 bytes).....                                                                                                                                                                                                                                                                                                     | 4   |
| 1.7. Exception Response Format .....                                                                                                                                                                                                                                                                                          | 5   |
| 1.8. Advantages of Modbus .....                                                                                                                                                                                                                                                                                               | 6   |
| 2. Software-In-the-Loop (SIL) Modbus & Serial Port.....                                                                                                                                                                                                                                                                       | 7   |
| 2.1. Port setup and choosing the command protocol .....                                                                                                                                                                                                                                                                       | 8   |
| 2.2. How to add Serial Ports to Your Computer and Connect Them as a Loop.....                                                                                                                                                                                                                                                 | 10  |
| 2.3. How to choose or write a new message.....                                                                                                                                                                                                                                                                                | 11  |
| 2.4. Calculation .....                                                                                                                                                                                                                                                                                                        | 14  |
| 2.4.1. Hex-to-Decimal and Decimal-to-Hex Conversion .....                                                                                                                                                                                                                                                                     | 14  |
| 2.4.2. Hex-to-Real and Real-to-Hex Conversion.....                                                                                                                                                                                                                                                                            | 14  |
| 2.4.3. Hex-to-String and String-to-Hex Conversion.....                                                                                                                                                                                                                                                                        | 15  |
| 2.5. About .....                                                                                                                                                                                                                                                                                                              | 16  |

## List of Abbreviations

|               |                                                                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SIL</b>    | Software-in-the-Loop.                                                                                                                           |
| <b>HIL</b>    | Hardware-in-the-Loop.                                                                                                                           |
| <b>PLC</b>    | Programmable Logic Controller.                                                                                                                  |
| <b>IEEE</b>   | Institute of Electrical and Electronics Engineers ( <a href="https://www.ieee.org">https://www.ieee.org</a> ).                                  |
| <b>Hex</b>    | Hexadecimal, a base-16 number system used in mathematics and computing.                                                                         |
| <b>Modbus</b> | Combining <b>Modicon</b> (the company that created the protocol) and <b>bus</b> (a communication pathway).                                      |
| <b>HMI</b>    | Human-Machine Interface.                                                                                                                        |
| <b>SCADA</b>  | Supervisory Control and Data Acquisition.                                                                                                       |
| <b>RTU</b>    | Remote Terminal Unit.                                                                                                                           |
| <b>CRC</b>    | Cyclic Redundancy Check, an error-detecting code used in digital networks and storage devices to spot accidental changes or corruption in data. |
| <b>RS-232</b> | Recommended Standard 232 serial communication interface.                                                                                        |
| <b>RS-485</b> | Recommended Standard 485 serial communication interface.                                                                                        |
| <b>ID</b>     | Identification number, such as a device or slave address.                                                                                       |
| <b>GND</b>    | Ground in electronics and electrical engineering.                                                                                               |
| <b>COM</b>    | Communication port, which is the traditional name for a serial port interface on a computer.                                                    |
| <b>LPT</b>    | Line Print Terminal: An older computing term for a parallel printer port on a computer (like LPT1).                                             |
| <b>PT100</b>  | Resistance Temperature Detector (RTD) used to measure temperature accurately.                                                                   |
| <b>A/D</b>    | Analog-to-Digital.                                                                                                                              |
| <b>ADC</b>    | Analog-to-Digital Converter.                                                                                                                    |
| <b>Min</b>    | Minimum.                                                                                                                                        |
| <b>Max</b>    | Maximum                                                                                                                                         |

## List of Figures

|                                                                                            |    |
|--------------------------------------------------------------------------------------------|----|
| <b>Figure 1: Message Frame</b> .....                                                       | 5  |
| <b>Figure 2: RS-485 Wiring Topology</b> .....                                              | 6  |
| <b>Figure 3: Program Interface</b> .....                                                   | 8  |
| <b>Figure 4: Find the available port in Windows Device Manager</b> .....                   | 9  |
| <b>Figure 5: COM Port and Baud Rate Configuration</b> .....                                | 9  |
| <b>Figure 6: Using and Setting Up Virtual Serial Port COM Port</b> .....                   | 10 |
| <b>Figure 7: Message Selection and Message Configuration</b> .....                         | 13 |
| <b>Figure 8: Hex-to-Decimal and Decimal-to-Hex Conversion</b> .....                        | 14 |
| <b>Figure 9: 4 Byte Hex(IEEE)-to-Real and Real-to-4 Byte Hex(IEEE) Conversion</b> .....    | 14 |
| <b>Figure 10: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion</b> ..... | 14 |
| <b>Figure 11: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion</b> ..... | 15 |
| <b>Figure 12: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion</b> ..... | 15 |
| <b>Figure 13: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion</b> ..... | 15 |
| <b>Figure 14: Hex-to-String and String-to-Hex Conversion</b> .....                         | 15 |
| <b>Figure 15: Software About</b> .....                                                     | 16 |

# **1. Introduction**

## 1.1. Introduction

The **Software-In-the-Loop (SIL) and Serial Port Calculator** is a software tool for PLC programmers and engineers who need a simulation environment with serial communication ports (RS-232/RS-485) to respond on behalf of real devices. It provides practical utilities for industrial communication, including:

- **Hex-to-decimal and decimal-to-hex conversion**
- **2-byte and 4-byte IEEE (floating-point) value calculation and conversion**
- **Hex-to-string and string-to-hex conversion**
- **Simulate a machine and respond to preset serial port command (SIL)**
- **Support for Modbus protocol**

## 1.2. Overview of the Modbus Protocol

**Modbus** is a widely used **industrial communication protocol** originally developed by **Modicon (now Schneider Electric)** in 1979. It is designed for reliable communication between **industrial electronic devices**, such as **PLCs, sensors, actuators, HMIs, and SCADA systems**.

Modbus is valued for its **simplicity, openness, and robustness**, making it one of the most commonly implemented protocols in industrial automation.

## 1.3. Modbus Communication Model

Modbus follows a **master–slave** (also referred to as **client–server**) architecture:

- The **Master (Client)** initiates all communication requests.
- The **Slave (Server)** devices respond to the master's requests.
- Slave devices never communicate with each other directly.

Each slave device has a **unique address**, allowing the master to communicate with multiple devices on the same network.

## 1.4. Modbus RTU

- Uses **binary data format**
- Communicates over **serial links** such as RS-485 or RS-232
- High efficiency and compact data frames
- Widely used in industrial environments

## 1.5. Modbus Data Model

Modbus organizes data into four basic data types:

- **Coils** (Read/Write – Digital Outputs)
- **Discrete Inputs** (Read-only – Digital Inputs)
- **Input Registers** (Read-only – Analog Inputs)
- **Holding Registers** (Read/Write – Analog Values and Parameters)

Each data type is accessed using specific **function codes** defined by the Modbus standard.

## 1.6. Modbus RTU Message Frame

A standard **Modbus RTU** frame consists of the following fields:

| Address | Function Code | Data | CRC |

### 1.6.0. Field Descriptions:

#### 1.6.1. Address (1 byte)

- Identifies the target slave device
- Valid range: **1–247**
- Address **0** is reserved for broadcast messages (no response)

#### 1.6.2. Function Code (1 byte)

Defines the action requested by the master.

##### Function Code Description

|           |                          |
|-----------|--------------------------|
| 01        | Read Coils               |
| 02        | Read Discrete Inputs     |
| 03        | Read Holding Registers   |
| 04        | Read Input Registers     |
| 05        | Write Single Coil        |
| 06        | Write Single Register    |
| 15 (0x0F) | Write Multiple Coils     |
| 16 (0x10) | Write Multiple Registers |

#### 1.6.3. Data Field (N bytes)

Contains parameters or returned values, depending on the function code.

#### 1.6.4. CRC (2 bytes)

- Cyclic Redundancy Check for error detection
- Sent as **low byte first, then high byte**

##### Example:

This example is a real Modbus message from the ADAM-4017+ module manufactured by Advantech. The module is configured to operate using the Modbus protocol.

Send → 10 04 00 00 00 08 F2 8D

Message Structure:

1. **Slave ID:** Identifies the target device.  
The first hexadecimal byte is 10, which represents the slave (node) address in hexadecimal (**17 in decimal**).
2. **Function Code:** Specifies the operation requested by the master (e.g., read or write).  
The second hexadecimal byte is 04, indicating the **Read Input Registers** function.
3. **Data:** Defines the registers to be accessed.
  - The third and fourth hexadecimal bytes (00 00) specify the **starting register address**.
  - The fifth and sixth hexadecimal bytes (00 08) indicate the **number of registers to be read**.
4. **CRC (Cyclic Redundancy Check):** Used for error detection.  
The seventh and eighth hexadecimal bytes (F2 8D) represent the **message CRC value**.

Received → 10 03 10 7F FF 80 02 7F FF 7F FF 7F FC 7F FE 7F FE 7F FC 54 82

Message Structure:

1. **Slave ID:** Identifies the responding device  
The first hexadecimal byte is 10, which represents the slave (node) address in hexadecimal (**17 in decimal**).
2. **Function Code:** Specifies the operation performed.  
The second hexadecimal byte is 03, indicating the **Read Holding Registers** function.
3. **Data:** Contains the returned register values.
  - The third hexadecimal byte (10) specifies the **byte count**, indicating **16 bytes** of data follow.
  - The fourth through nineteenth hexadecimal bytes (7F FF 80 02 7F FF 7F FF 7F FC 7F FE 7F FE 7F FC) represent **eight 2-byte register values** returned by the slave.

4. **CRC (Cyclic Redundancy Check):** Used for error detection. The twentieth and twenty-first hexadecimal bytes (54 82) represent the **message CRC value**.

The message structure for other Modbus devices may be similar; however, details differ, including how real data types are represented, which typically use IEEE floating-point format.

## 1.7. Exception Response Format

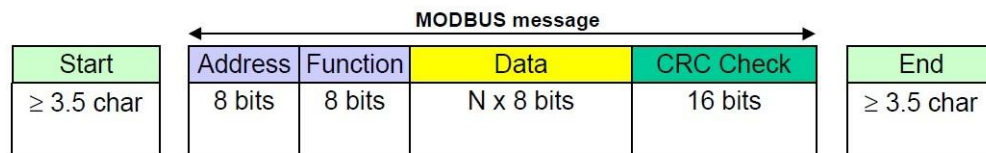
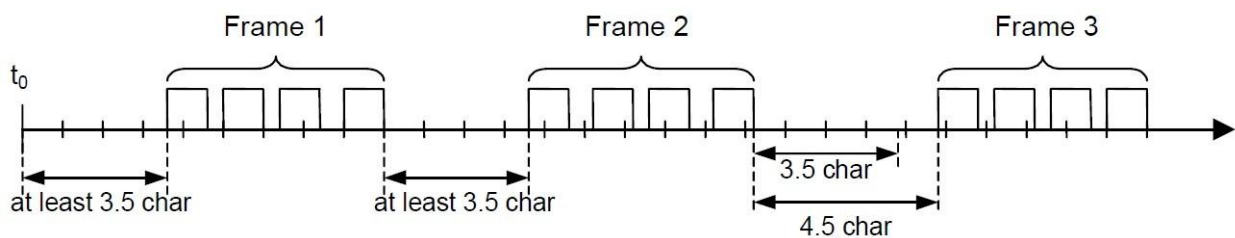
If an error occurs, the slave responds as follows:

| Address | Function Code + 0x80 | Exception Code | CRC |

### Common Exception Codes:

- 01 – Illegal Function
- 02 – Illegal Data Address
- 03 – Illegal Data Value
- 04 – Slave Device Failure

| Slave Address | Function Code | Data                | CRC                       |
|---------------|---------------|---------------------|---------------------------|
| 1 byte        | 1 byte        | 0 up to 252 byte(s) | 2 bytes<br>CRC Low CRC Hi |



RTU Message Frame

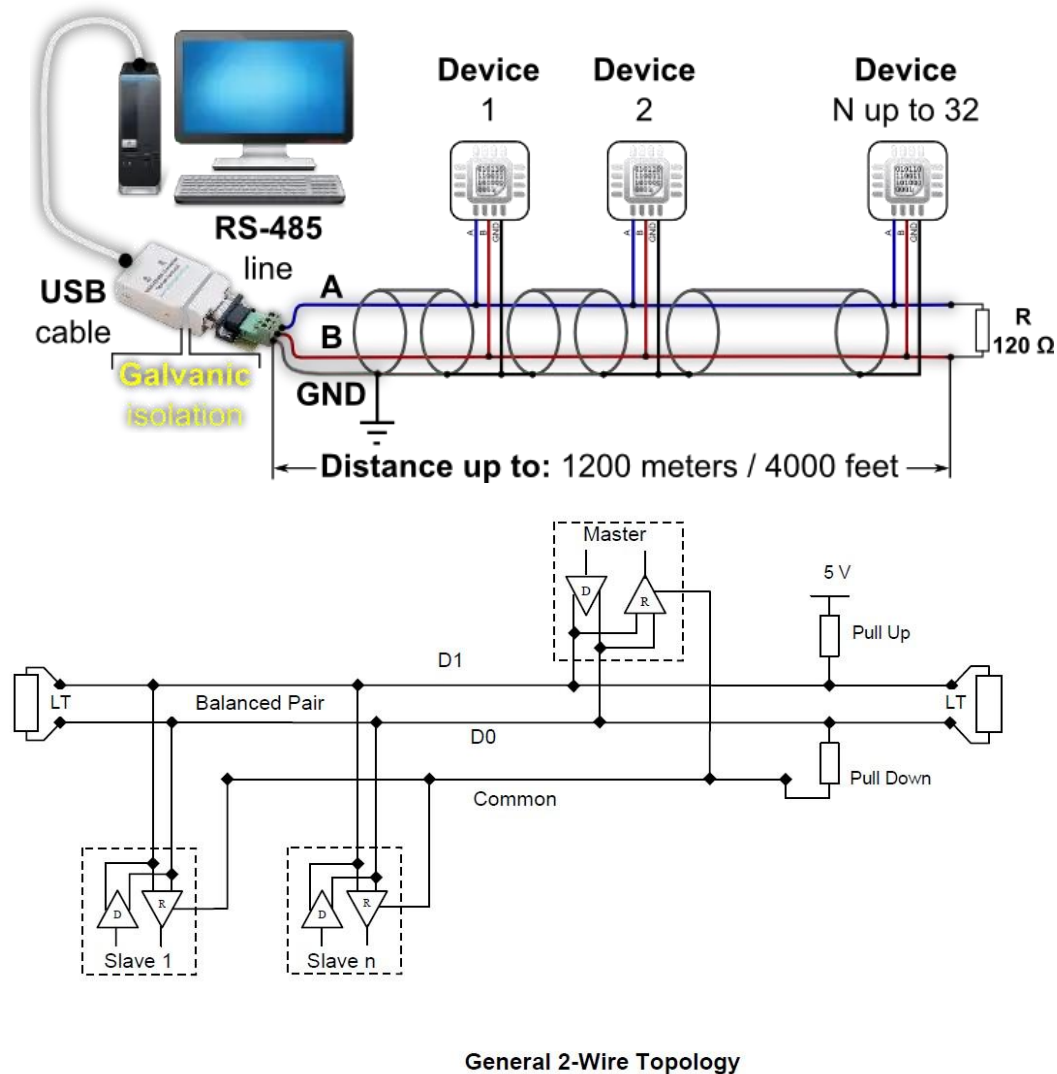
**Figure 1: Message Frame**

## 1.8. Advantages of Modbus

- Open and license-free protocol
- Easy to implement and troubleshoot
- Supported by a wide range of manufacturers
- Reliable communication over long distances (RS-485)

### Typical Applications

- Industrial automation systems
- PLC-to-device communication
- Energy management systems
- Building automation
- Data acquisition and monitoring



**Figure 2: RS-485 Wiring Topology**

## **2. Software-In-the-Loop (SIL) Modbus & Serial Port**

## 2.1. Port setup and choosing the command protocol

This software enables users to simulate a device connected through a serial port for a third-party application, acting on behalf of the real machine. It is particularly useful when the actual device is not available in the programming environment or when the machine requires significant power, space, or special operating conditions. The software allows the user to:

- Configure serial port settings
- Select or build the appropriate response to received commands
- Calculate the Modbus checksum (CRC) according to the Modbus RTU standard
- Wait a predefined number of milliseconds and then read the response
- Convert hexadecimal values to decimal
- Convert 4-byte hexadecimal values (IEEE floating-point format) to real numbers
- Convert 2-byte hexadecimal values to real numbers

Software in the Loop(SIL)/Modbus RTU Serial Port

Message: ADAM 4017+M Protocol: Modbus COM Port: COM2

BaudRate: 9600 Delay Between Send and Receive[mSec]: 50

Send: 10 04 00 00 00 08 F6 4D

Receive: 10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc E8 D0

Received: 10 04 00 00 00 08 F2 8D -->

Sending: 10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc 54 82 -->

Message: The message was not received!

Hex 2 Decimal Conversion

2 Byte Hex: 7F FF Decimal Value: 32767

Hex 2 Real Conversion

Real 4 Byte (IEEE) 42 D7 37 84 Real Value: 107.608429

Hex 2 String

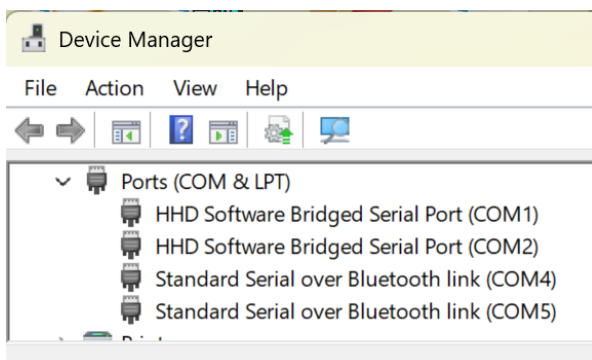
Hex: 23 31 30 0D String: #10.

About Close

**Figure 3: Program Interface**

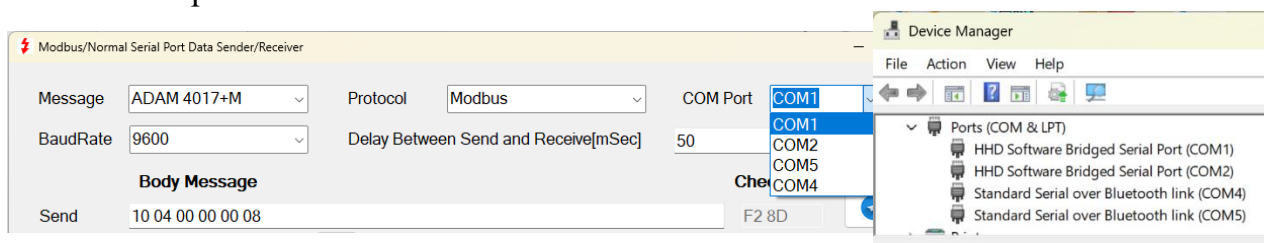
## How to Identify a Serial Port on Your Computer

- Open the **Device Manager** application.
- Expand the **Ports (COM & LPT)** section.
- If a serial port is available, it will be listed there, as shown in the snapshot below.

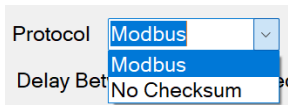


*Figure 4: Find the available port in Windows Device Manager*

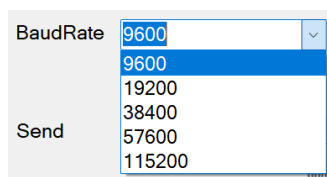
The program automatically detects existing COM ports, and the user can choose one of the available ports.



How to change the Protocol



How to change the Baud Rate



*Figure 5: COM Port and Baud Rate Configuration*

The latest settings are saved in the “**ModbusConverSIL.ini**” file and are restored the next time the software is opened.

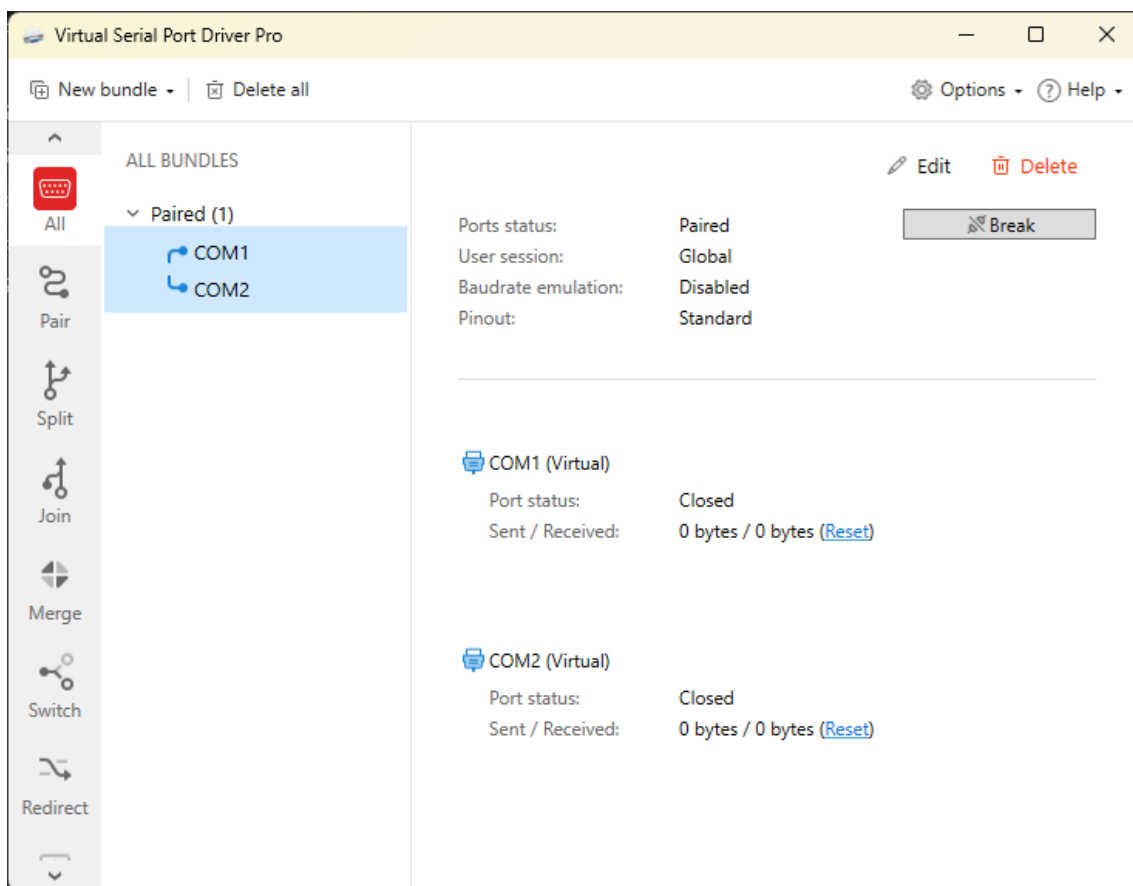
## 2.2. How to add Serial Ports to Your Computer and Connect Them as a Loop

When a device is connected to a computer through a serial port, it is often necessary to access real data for software development and testing. However, the actual device may not always be available at the development location, or the required operating conditions may be difficult to reproduce.

For example, the software may need to be tested under special conditions, such as very low temperatures or other environments that are not easily achievable in the development laboratory.

In such situations, a Software-In-the-Loop (SIL) approach can be used to simulate the behavior of the real device through software, allowing developers to test communication, data processing, and control logic without requiring the physical hardware.

A third-party tool such as “Virtual Serial Port Driver Pro” can be used to create a pair of connected virtual serial ports on the computer for simulation purposes.



*Figure 6: Using and Setting Up Virtual Serial Port COM Port*

## 2.3. How to choose or write a new message

The operator can configure up to 10 messages in the “**MessageSIL.ini**” file, which contains both the message protocol definition and the corresponding display name used by the software. The package includes predefined messages for three Advantech devices: “**ADAM-4017**” for reading eight voltage or current channels, “**ADAM-4015**” for reading six **PT100** temperature sensors, and “**ADAM-4018**” for reading eight **thermocouple** temperature sensors. For each device, two lines must be defined in the file. The first line represents the message sent by the computer to the device. When the **SIL** software receives this message, it recognizes the command and prepares the corresponding response. In other words, the message sent from the computer must first be received by the SIL device in order to generate the appropriate response. The structure of the message is shown in the following sample of the “**MessageSIL.ini**” file.

```
10 04 00 00 00 08;Modbus;30;ADAM 4017+;Advantech Adam 4017+ Modbus protocol for A/D
10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc; We must add Modbus protocol CRC

13 04 00 00 00 06;Modbus;30;ADAM 4015;Advantech Adam 4015 Modbus protocol for PT100 temperature Sensor
13 03 0c 5f 21 5D 63 58 5A ff ff ff ff ff ff;We must add Modbus protocol CRC

16 04 00 00 00 08;Modbus;30;ADAM 4018;Advantech Adam 4018 Modbus protocol for thermocouple Sensor
16 03 10 5f 21 5D 63 58 5A ff ff ff ff ff ff ff ff;We must add Modbus protocol CRC

23 31 30 0D;No Checksum;30;ADAM 4017;Advantech Adam 4017 Advantech protocol for A/D
3E 2B 37 2E 32 31 31 31 2B 37 2E 32 35 36 37 2B 37 2E 33 31 32 35 2B 37 2E 31 30 30 30 2B 37!
!2E 34 37 31 32 2B 37 2E 32 35 35 35 2B 37 2E 31 32 33 34 2B 37 2E 35 36 37 38 !
!20 0D; >+7.2111+7.2567+7.3125+7.1000+7.4712+7.2555+7.1234+7.5678 (cr)

23 31 33 0D;No Checksum;30;ADAM 4015;Advantech Adam 4015 Advantech protocol for PT100 temperature
Sensor
3E 2B 32 32 2E 31 31 31 2B 32 33 2E 32 32 32 2B 32 34 2E 33 33 33 2B 32 35 2E 34 34 34 !
!2B 32 36 2E 35 35 35 2B 32 37 2E 38 38 38 20 0D; >+22.111+23.222+24.333+25.444+26.555+27.888 (cr)

23 31 36 0D;No Checksum;30;ADAM 4018;Advantech Adam 4018 Advantech protocol for thermocouple Sensor
3E 2B 32 32 2E 31 31 31 2B 32 33 2E 32 32 32 2B 32 34 2E 33 33 33 2B 32 35 2E 34 34 34 2B!
!32 36 2E 35 35 35 2B 32 37 2E 38 38 38 2B 32 38 2E 39 39 39 2B 32 39 2E !
!31 31 31 20 0D; >+22.111+23.222+24.333+25.444+26.555+27.888+28.999+29.111 (cr)
```

Each message is defined using two lines in the configuration file. Each part of a line is separated by a semicolon (;), as shown below.

In 1<sup>st</sup> line (Command Received by **SIL**):

- The **first part** defines the message that is received by the Software-In-the-Loop (SIL), when the corresponding command is sent from the external program. The message is written in **hexadecimal format**. Each ASCII character is represented by **two hexadecimal digits** ranging from **00** to **FF**. For example, **10** hex(hexadecimal) is equal to 16 in **decimal**. Each byte is separated by a space.
- The **second part** defines the **message protocol**. Two options are available: “**Modbus**” or “**No Checksum**”. If “**Modbus**” is selected, the checksum (CRC) is calculated and automatically appended to the message before transmission.

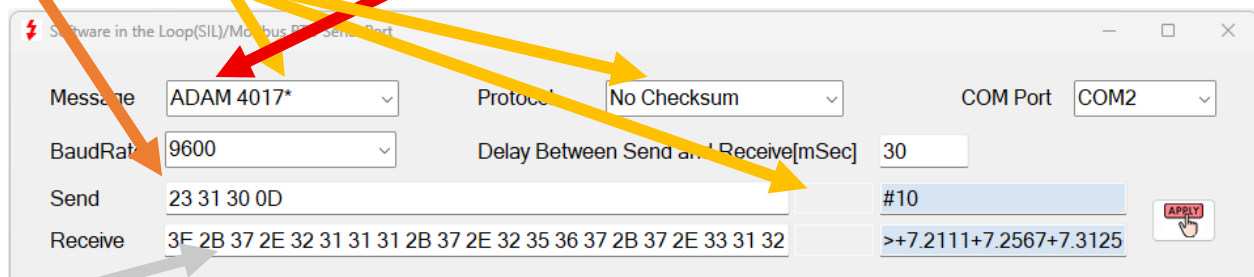
- In 2<sup>nd</sup> line (Response Sent by **SIL**):

- If “**Modbus**” is selected in the 2<sup>nd</sup> part, the letter “**M**” is appended to the end of the device name (in the 4<sup>th</sup> part), and if “**No Checksum**” is selected, an asterisk (“**\***”) is appended to the device name.

10 04 00 00 00 08;Modbus;50;ADAM 4017+;Advantech Adam 4017+ Modbus protocol for A/D

10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc; We must add Modbus protocol CRC

23 31 30 0D; No Checksum; 30; ADAM 4017; Advantech Adam 4017 Advantech protocol for A/D



3E 2B 37 2E 32 31 31 31 2B 37 2E 32 35 36 37 2B 37 2E 33 31 32 35 2B 37 2E 31 30 30 30 2B 37 2E 34 37 31  
32 2B 37 2E 32 35 35 35 2B 37 2E 31 32 33 34 2B 37 2E 35 36 37 38 20 0D;  
>+7.2111+7.2567+7.3125+7.1000+7.4712+7.2555+7.1234+7.5678 (cr)

**Figure 7: Message Selection and Message Configuration**

All sent and received data are logged in the “**ModbusConverSIL.txt**” file.

Sample content of the “**ModbusConverSIL.txt**” file is shown below:

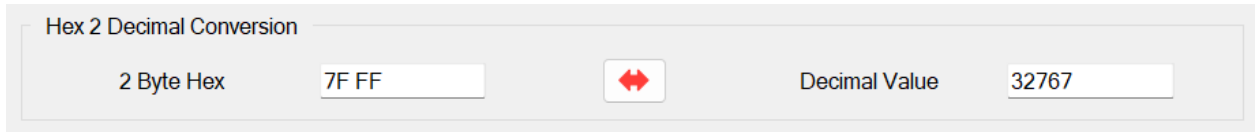
This text file was generated by ModbusConver.exe and developed by Babak Hajari. (Caution: Do not delete this line.)

```
2026/03/08 11:08
Message => Check CRC was OK. the data from
Send M => 10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc 54 82
Received => 10 04 00 00 00 08 F2 8D
Send => 3E 2B 37 2E 32 31 31 31 2B 37 2E 32 35 36 37 2B 37 2E 33 31 32 35 2B 37 2E 31 30 30 30 2B 37
2E 34 37 31 32 2B 37 2E 32 35 35 35 2B 37 2E 31 32 33 34 2B 37 2E 35 36 37 38 20 0D 8C 40
```

## 2.4. Calculation

### 2.4.1. Hex-to-Decimal and Decimal-to-Hex Conversion

In this section, you can convert **2-byte hexadecimal values** to **decimal values**, and vice versa.

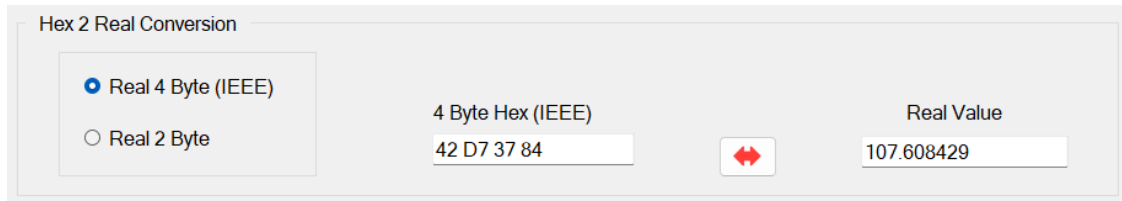


The interface is titled "Hex 2 Decimal Conversion". It features a "2 Byte Hex" input field containing "7F FF", a red double-headed arrow button, a "Decimal Value" output field containing "32767", and a "Real Value" input field which is currently empty.

*Figure 8: Hex-to-Decimal and Decimal-to-Hex Conversion*

### 2.4.2. Hex-to-Real and Real-to-Hex Conversion

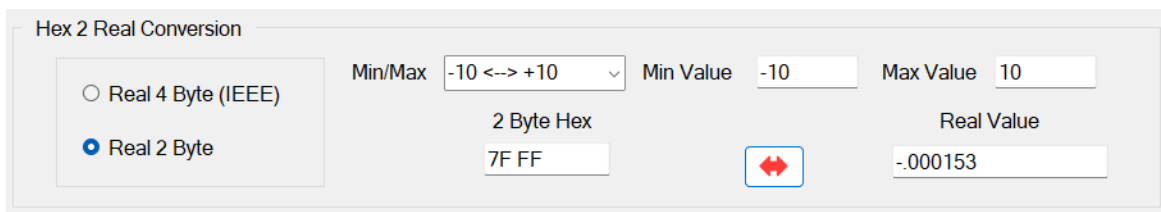
In this section, you can convert **4-byte hexadecimal values (in IEEE floating-point format)** to **real values** and vice versa.



The interface is titled "Hex 2 Real Conversion". It has two radio buttons: "Real 4 Byte (IEEE)" (selected) and "Real 2 Byte". The "4 Byte Hex (IEEE)" input field contains "42 D7 37 84", followed by a red double-headed arrow button, and a "Real Value" output field containing "107.608429".

*Figure 9: 4 Byte Hex(IEEE)-to-Real and Real-to-4 Byte Hex(IEEE) Conversion*

In this section, you can convert **2-byte hexadecimal values** into **real values** using defined **minimum and maximum limits**. For example, if the minimum value is **-10 [volts]** and the maximum value is **10 [volts]**, the corresponding real value is calculated based on the hexadecimal input.



The interface is titled "Hex 2 Real Conversion". It has two radio buttons: "Real 4 Byte (IEEE)" and "Real 2 Byte" (selected). Above the input fields, there are controls for "Min/Max" (a dropdown menu showing "-10 <--> +10"), "Min Value" (input field with "-10"), and "Max Value" (input field with "10"). The "2 Byte Hex" input field contains "7F FF", followed by a red double-headed arrow button, and a "Real Value" output field containing "-.000153".

*Figure 10: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion*

For example, if the minimum value is **4 [milli-amperes]** and the maximum value is **20 [milli-amperes]**, the corresponding real value is calculated based on the hexadecimal input.

The screenshot shows the 'Hex 2 Real Conversion' window. On the left, 'Real 2 Byte' is selected. The 'Min/Max' dropdown is set to '+4 <--> +20'. The 'Min Value' field contains '4' and the 'Max Value' field contains '20'. The '2 Byte Hex' field contains '7F FF'. A red double-headed arrow button is between the hex and real value fields. The 'Real Value' field displays '11.999878'.

*Figure 11: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion*

For example, if the minimum value is **-50 [°C]** and the maximum value is **150 [°C]**, the corresponding real value is calculated based on the hexadecimal input.

The screenshot shows the 'Hex 2 Real Conversion' window. On the left, 'Real 2 Byte' is selected. The 'Min/Max' dropdown is set to '-50 <--> 150'. The 'Min Value' field contains '-50' and the 'Max Value' field contains '150'. The '2 Byte Hex' field contains '7F FF'. A red double-headed arrow button is between the hex and real value fields. The 'Real Value' field displays '49.998474'.

*Figure 12: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion*

If the minimum and maximum values are not listed among the predefined values, you can enter them directly in the Min and Max fields.

The screenshot shows the 'Hex 2 Real Conversion' window. On the left, 'Real 2 Byte' is selected. The 'Min/Max' dropdown is empty. The 'Min Value' field contains '0' and the 'Max Value' field contains '1000'. The '2 Byte Hex' field contains '7F FF'. A red double-headed arrow button is between the hex and real value fields. The 'Real Value' field displays '499.99237'.

*Figure 13: 2 Byte Hex(16Bit)-to-Real and Real-to-2 Byte Hex(16Bit) Conversion*

### 2.4.3. Hex-to-String and String-to-Hex Conversion

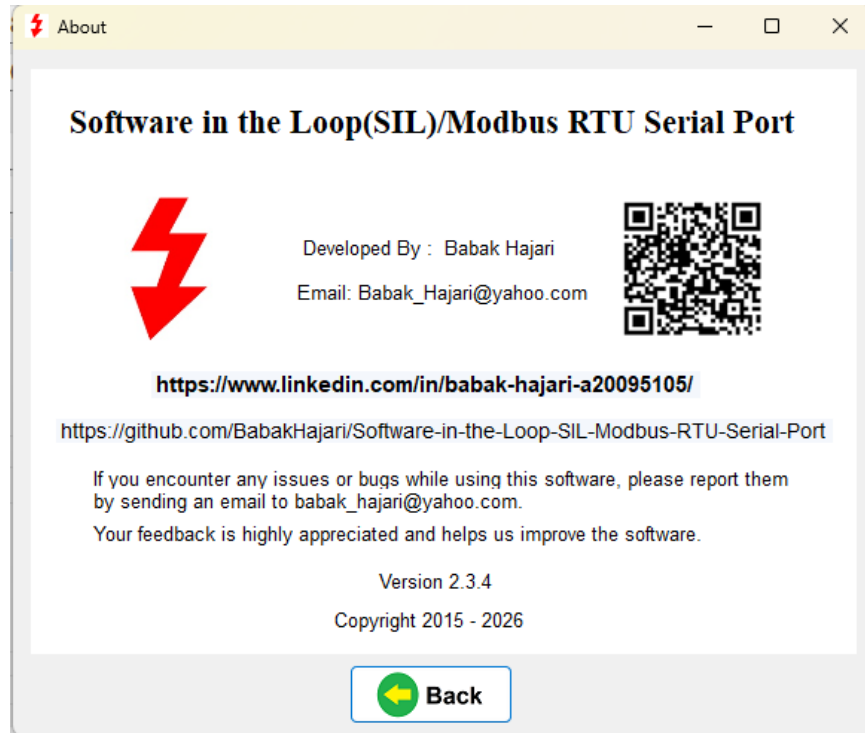
In this section, you can convert **hexadecimal values to a string** and vice versa.

The screenshot shows the 'Hex 2 String' window. The 'Hex' field contains '23 31 30 0D'. The 'String' field contains '#10.'. A red double-headed arrow button is on the right side of the interface.

*Figure 14: Hex-to-String and String-to-Hex Conversion*

## 2.5. About

By pressing the “**About**” button, information about the package and its developer is displayed in a new window, as shown below.



*Figure 15: Software About*